Creating and benchmarking Learned DB Components using LLM

Preksha Shah

Master's Thesis
FB Informatik

April 28, 2026

RPTU
Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Agenda

1. Motivation
2. Pipeline
3. LLM Query Generation
4. MSCN Architecture
5. Active Learning
6. Experimental Results
7. Conclusion
8. References

Why Learned Cardinality Estimation?

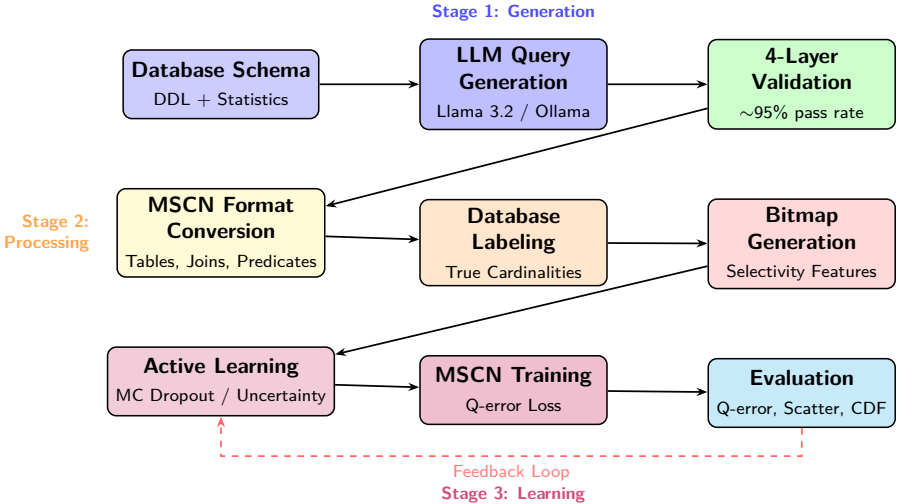
The Problem

- Bad cardinality estimates lead to catastrophic join orders
- Histograms and independence assumptions fail on correlated data
- Sampling is expensive at runtime and misses rare groups

The Learned Approach

- Deep learning models (MSCN) learn correlations directly from data
- Outperform traditional mathematical estimators
- **But:** Require massive labeled workloads (queries + true cardinalities)
- Obtaining labels via query execution is expensive and hard to scale

Pipeline: From Schema to Trained Estimator



LLM-based Query Generation

Schema-aware prompting with diversity control

Prompt Strategy:

- **Input:** Schema DDL + column statistics
- **Output:** JSON array of SQL queries
- **Diversity hints:** Rotate across 5 structural profiles
- **Batched generation:** Manages token limits, retries

4-Layer Validation:

- Keyword → Parse → Schema → Format
- ∼95% pass rate

Strict Constraints:

- `SELECT COUNT(*)` only
- 1–3 tables with proper aliases
- Joins on defined foreign keys
- Simple comparisons (=, <, >) with `AND`
- Prohibited: `OR`, `IN`, `LIKE`, subqueries

MSCN: Multi-Set Convolutional Network

Set-based encoding with masked pooling

Query Encoding:

- **Tables:** One-hot + bitmap sample
- **Predicates:** Column + operator + normalized value
- **Joins:** One-hot of join condition

Architecture:

- Parallel 2-layer MLPs per set
- Masked mean pooling (ignores padding)
- Concatenate → output MLP → sigmoid
- Loss: Mean Q-error (scale-invariant)

Active Learning for Cardinality Estimation

The Labeling Bottleneck: Every label requires executing a query on a live database. Complex joins can take seconds each. Labeling thousands of queries takes hours.

Standard Supervised

1. Generate massive dataset
2. **Label everything**
3. Train model once

Wasteful: labels redundant, easy queries

Active Learning

1. Label small seed (20%)
2. Train initial model
3. Score unlabeled queries by uncertainty
4. **Label only the hardest**
5. Retrain and repeat

Efficient: focuses budget where it matters

Acquisition Strategies

How to decide which queries to label next

1. **Random Sampling** — Baseline. No scoring, just diversity from the pool.
2. **Uncertainty Sampling** — Select queries where sigmoid output is closest to 0.5:

$$\alpha_{\text{uncertainty}}(q) = -\left|f_\theta(q) - 0.5\right|$$

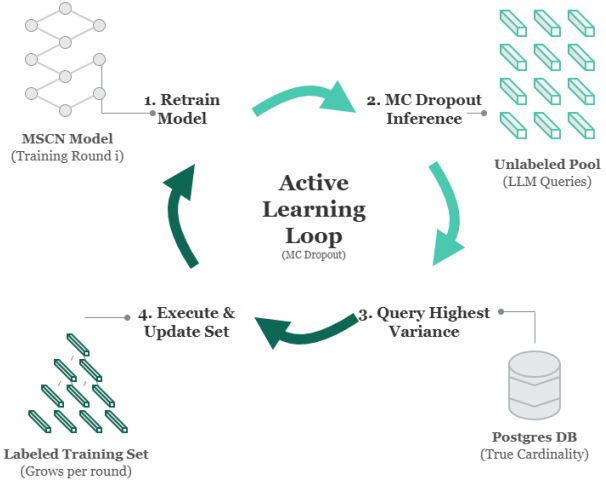
Cheap (single forward pass), but ignores diversity among selected queries.

3. **Monte Carlo Dropout** — Keep dropout active, run $T = 25$ forward passes:

$$\alpha_{\text{MC}}(q) = \text{Var}\left(\{\hat{y}^{(t)}\}_{t=1}^{T}\right)$$

High variance = fragile representation = most informative to label. Richer signal than plain uncertainty; modest overhead vs. database execution cost.

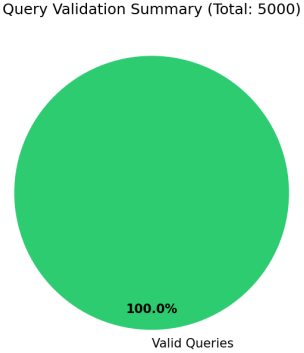
The Active Learning Loop



Experimental Setup & Data Quality

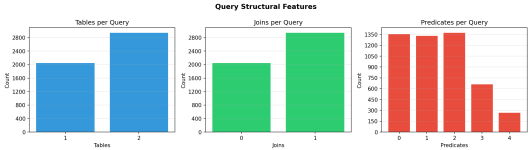
Configuration:

- **Database:** IMDB benchmark
- **LLM:** Llama 3.2 via Ollama
- **Model:** MSCN (hidden=256, dropout=0.1)
- **AL:** MC Dropout, $T = 25$, 5 rounds
- **Metric:** Q-error

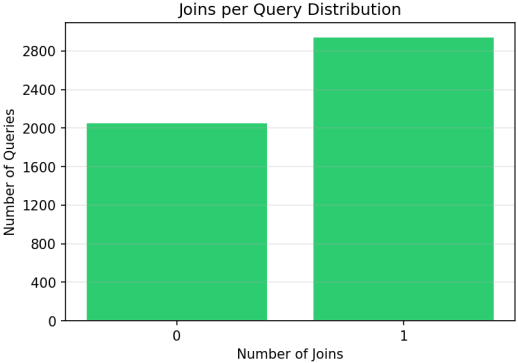


Query validation success rate across generation batches

Workload Diversity



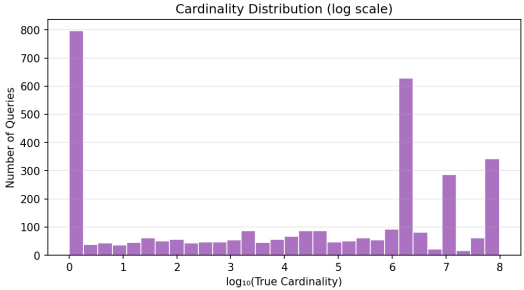
Structural feature distributions



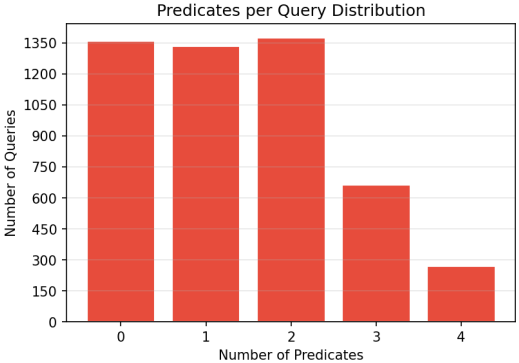
Join count distribution

Diversity hints produce a balanced mix of single-table, two-table, and three-table queries with varied predicate counts.

Cardinality & Predicate Coverage



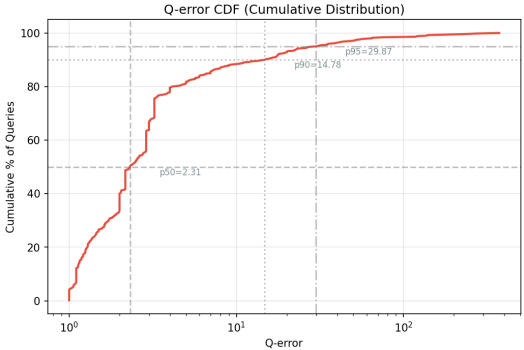
True cardinality distribution (log scale)



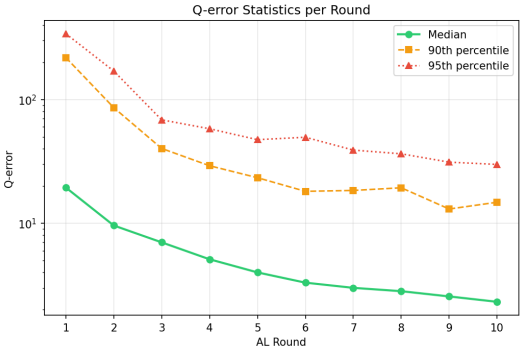
Predicate count distribution

Generated workload spans cardinalities from single-digit to millions, covering the full range needed for robust estimation.

Estimation Accuracy: Q-error Analysis



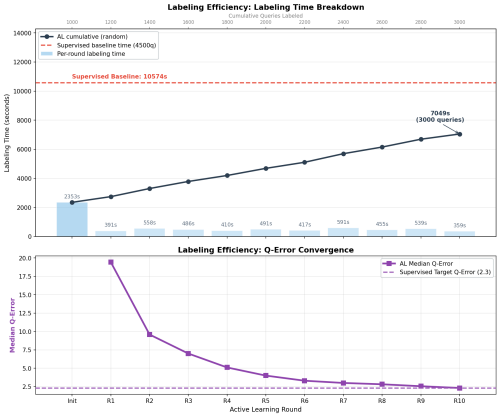
Q-error CDF: majority within factor of 2–5



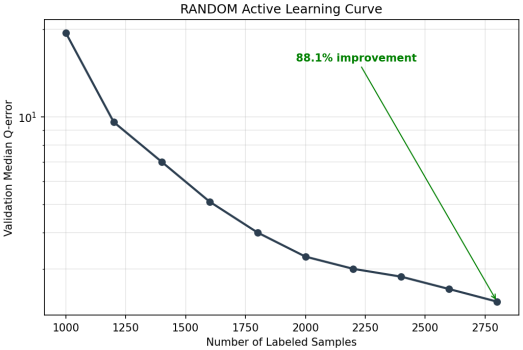
Q-error statistics per AL round

Median, mean, and tail Q-errors all decrease with each active learning round.

Active Learning: Labeling Efficiency



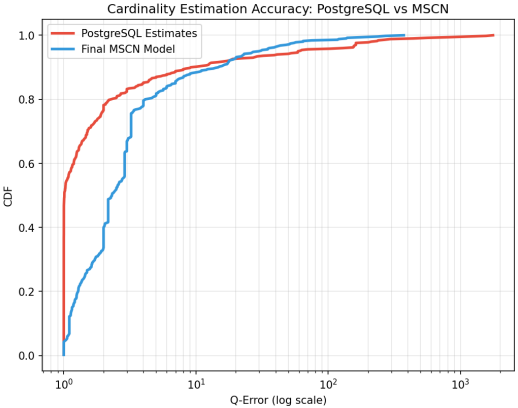
AL achieves competitive accuracy with fewer labels



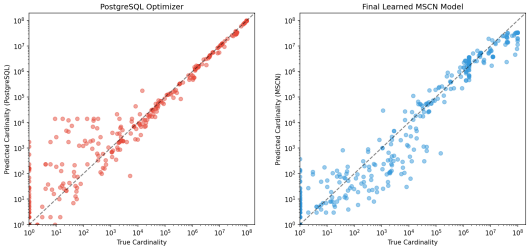
Learning curve across AL rounds

Intelligent query selection reduces labeling cost by ∼40% while maintaining estimation quality.

MSCN vs. PostgreSQL Optimizer



Q-error CDF: MSCN vs. PostgreSQL



Predicted vs. actual: closer to diagonal = better

The MSCN model trained with active learning matches or outperforms PostgreSQL's built-in estimator.

Conclusion

What we showed:

- LLMs generate diverse, realistic SQL workloads from schema alone ($\sim$95% valid)
- Active learning with MC Dropout reduces labeling cost by $\sim$40%
- The trained MSCN matches PostgreSQL's built-in estimator
- The pipeline is schema-agnostic and fully automated

Future Directions:

- Extend to richer SQL fragments (OR, LIKE, GROUP BY)
- Cross-schema transfer learning
- Integration with live query optimizers
- Hybrid acquisition: uncertainty + diversity

References I

Kipf, A., Kipf, T., Radke, B., Leis, V., Boncz, P., & Kemper, A. (2019).
Learned Cardinalities: Estimating Correlated Joins with Deep Learning.
CIDR 2019.

Wang, X., Qu, C., Wu, W., Wang, J., & Zhou, Q. (2021).
Are We Ready For Learned Cardinality Estimation?
Proc. VLDB Endow., 14(9), 1640–1654.

Davitkova, A. & Michel, S. (2025).
Automated Training of Learned Database Components with Generative AI.
CoRR, abs/2512.20271.

References II

Gal, Y. & Ghahramani, Z. (2016).
Dropout as a Bayesian Approximation.
ICML 2016, 1050–1059.

Thank you

Preksha Shah
qar12xun@rptu.de
RPTU Kaiserslautern